Labo2: Race conditions

Arthur Valentin, Joeri Sevens

November 15, 2024

Overview

In dit labo onderzoeken we drie verschillende methoden voor het vinden van het maximum in een 1D-array, met behulp van zowel de CPU als de GPU. Concreet implementeren we maximale detectie met behulp van: een geneste for-loop op de CPU, een GPU-kernel met een atomic operation, en een GPU-kernel die reduction toepast. Door deze benaderingen te vergelijken, verkennen we de verschillen in prestaties en efficiëntie tussen CPU- en GPU-implementaties.

Methodology

De nested for-loop op de CPU doorloopt de array en vergelijkt steeds de huidige waarde met de volgende, waarbij de grootste waarde wordt bijgehouden.

Vervolgens wordt er met behulp van een GPU-kernel de maximale waarde in een array gevonden via atomicMax. Elke thread berekent zijn positie in de array en vergelijkt de waarde op die plek met het huidige maximum, dat veilig wordt bijgewerkt met atomicMax. Na uitvoering van de kernel wordt het berekende maximum teruggehaald.

Uiteindelijk vindt de GPU-kernel het maximum in de array via een reduction-methode. In elke iteratie wordt een offset op basis van een macht van 2 bepaald, afhankelijk in welke stap dat we ons bevinden. Elke thread vergelijkt elementen die zich op een afstand van 2^j van elkaar bevinden, waarbij j de iteratienummer is. De thread slaat de grootste waarde op tussen de twee elementen. De kernel wordt herhaaldelijk aangeroepen, waarbij in elke ronde de helft van de elementen vergeleken en gereduceerd wordt, totdat het maximum bovenaan de array staat.

We hebben deze aanpak ook op een tweede manier uitgevoerd, waarbij de for-loop nu volledig op de GPU draait. In de GPU-kernel wordt de array parallel doorlopen, en elke thread vergelijkt waarden met een exponentieel groeiende afstand (2^j). De max functie bepaalt de grootste waarde per iteratie, en ___syncthreads() zorgt voor synchronisatie tussen threads. In de main-functie wordt de array naar de GPU gekopieerd, de kernel uitgevoerd, en het resultaat teruggehaald om het maximum te tonen. Alles gebeurt nu op de GPU zonder tussenkomst van de CPU.

Conclusion

We hebben geleerd om met race condities om te gaan.

Is the result correct?

Onze resultaten zijn correct, we krijgen steeds de grootste waarde van onze array terug.

How does time increase as a function of input size?

Op basis van de uitgevoerde metingen kunnen we concluderen dat de verwerkingstijd, afhankelijk van de arraygrootte, geleidelijk toeneemt. Voor de ”pure GPU”-benadering zien we een lichte stijging in de uitvoeringstijd naarmate de array groter wordt.

Wanneer de CPU ook betrokken is in de berekening (”met CPU”-scenario), is de verwerkingstijd hoger dan in de ”pure GPU”-scenario’s. Dit impliceert dat de CPU hier een beperkende factor is, mogelijk vanwege minder efficiëntie in het parallel verwerken van data.

Tot slot, in de ”atomic max”-scenario’s de snelste manier, tenzij je met super grote inputs werkt dan zal de ’pure GPU’ sneller zijn.

Dit komt doordat atomaire operaties synchronisatie vereisen, wat enige overhead veroorzaakt, vooral bij grotere datasets.

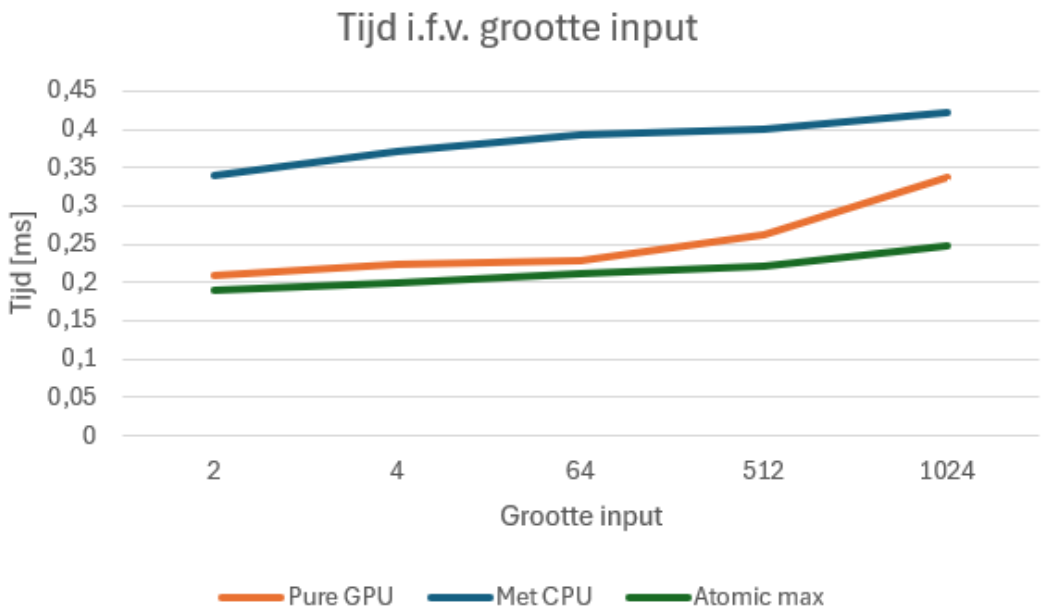


Figure 1: tijd meting

What happens if the input is larger than 2048 (how would you ensure correct calculation and are there additional synchronization steps)?

Zie code ”Code voor grotere arrays met GPU” onderaan. We splitsen de elementen in verschillende blokken op, dan berekenen we per blok het maximum en keren we in deze code wel eens terug naar de CPU voor de laatste stap die bestaat uit het maximum bepalen van alle maxima. We zouden dit ook opnieuw op de GPU kunnen doen door al deze waarden als een nieuwe array aan de GPU mee te geven of met shared memmory, maar dit is hier niet geïmplementeerd.

Consider calling the kernel with much less threads than there are data elements. For example, one tenth. How would you ensure all elements get processed?

Door striding toe te passen, kan één thread meerdere elementen verwerken. Hierbij overslaan de threads telkens een bepaald aantal stappen in de array om hun werk gelijkmatig te verdelen.

How would you tackle a 2D array?

Voor een 2D array zou ik eerst het maximum van elke rij zoeken, deze wordt door de onze code naar de eerste plaats gebracht van de array geschreven, waarna we het maximum van de 1ste kolom kunnen zoeken. Dit maximum is dan ook het globale maximum van de 2D array. Voor deze implementatie keren we echter eens terug naar de CPU wat relatief veel tijd kost. Dus een andere optie zou zijn om de 2D array te beschouwen als een lange 1D array en gewoon alle rijen achter elkaar te zetten, en dan puur op de GPU het maximum zoeken met de reeds geschreven code.

Code nested CPU

```cpp
#include <iostream>
using namespace std;

int main() {
    int N = 64;
    int A[N];

    for (int i = 0; i < N; i++) {
        A[i] = rand() % 100; // Random number between 0 and 99
    }

    for (int i = 0; i < N; i++) {
        cout << A[i] << " ";
        }
    cout << endl;

    printf("\n");

    int max = A[0];
    for (int i = 1; i < N; i++) {
        if (A[i] > max) {
            max = A[i];
        }
    }

    cout << "The maximum element is: " << max << endl;

    return 0;
}
```

Code atomic max

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void findMax(int* A, int* maxVal, int N) {
    int i =  blockIdx.x * blockDim.x + threadIdx.x; //berekend de plaats gebruik
        makende van de blocknr*groote block+ thread in block, anders doen de 2
        blokken exact het zelfde
    if (i < N) {
        atomicMax(maxVal, A[i]);
        }
}

int main() {
    int N = 1024;
    int A[N];
    int h_maxVal = INT_MIN;

    for (int i = 0; i < N; i++) {
        A[i] = rand() % 100; // Random number between 0 and 99
    }

    for (int i = 0; i < N; i++) {
        printf("%d␣", A[i]);
    }

    printf("\n");

    int *d_A, *d_maxVal;;
    cudaMalloc(&d_A, N * sizeof(int));      //maakt geheugen aan op gpu voor A
    cudaMalloc(&d_maxVal, sizeof(int));      //maakt geheugen aan op gpu voor B

    cudaMemcpy(d_A, A, N * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_maxVal, &h_maxVal, sizeof(int), cudaMemcpyHostToDevice);        //
        kopieert array A naar GPU

    int threadsPerBlock = 1024;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    cudaEvent_t start , stop ;
    cudaEventCreate (& start ) ;
    cudaEventCreate (& stop ) ;
    cudaEventRecord ( start ) ;

    findMax<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_maxVal, N);

    cudaDeviceSynchronize();
    cudaEventRecord ( stop ) ;
    cudaEventSynchronize(stop);
    float milliseconds ;
    cudaEventElapsedTime (& milliseconds , start, stop) ;

    cudaMemcpy(&h_maxVal, d_maxVal, sizeof(int), cudaMemcpyDeviceToHost);

    printf("Maximum␣value␣is:␣%d\n", h_maxVal);
    printf("time␣is:␣%f\n", milliseconds);

    cudaFree(d_A);
    cudaFree(d_maxVal);

    return 0;
}
```

Code reduction met CPU

```cpp
#include <stdio.h>
#include <cuda_runtime.h>
#include <cmath>

__global__ void findMax(int* A, int N, int j) {
    int i =  blockIdx.x * blockDim.x + threadIdx.x; //berekend de plaats gebruik makende van de blocknr*groote block+ thread in block, anders doen de 2 blokken exact het zelfde
    if (i < N){
        if (threadIdx.x % static_cast<int>(pow(2, j)) == 0) {
            A[i] = max(A[i], A[i+static_cast<int>(pow(2, j))]);
        }
    }
}

int main() {
    int N = 1024;
    int A[N];
    int result[N];

    for (int i = 0; i < N; i++) {
        A[i] = rand() % 100; // Random number between 0 and 99
    }

    for (int i = 0; i < N; i++) {
        printf("%d␣", A[i]);
    }
    printf("\n");

    int *d_A;
    cudaMalloc(&d_A, N * sizeof(int));      //maakt geheugen aan op gpu voor A
    cudaMemcpy(d_A, A, N * sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 1024;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    cudaEvent_t start , stop ;
    cudaEventCreate (& start ) ;
    cudaEventCreate (& stop ) ;
    cudaEventRecord ( start ) ;

    for (int i = 0; i < log2(N); i++){
        findMax<<<blocksPerGrid, threadsPerBlock>>>(d_A, N, i);
        cudaDeviceSynchronize();
    }

    cudaEventRecord ( stop ) ;
    cudaEventSynchronize(stop);
    float milliseconds ;
    cudaEventElapsedTime (& milliseconds , start, stop) ;

    cudaMemcpy(result, d_A, N * sizeof(int), cudaMemcpyDeviceToHost);        // kopieert array GPU naar B

    printf("Maximum␣value␣is:␣%d\n", result[0]);
    printf("time␣is:␣%f\n", milliseconds);

    cudaFree(d_A);

    return 0;
}
```

Code reduction met puur GPU

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <math_constants.h>

__global__ void findMax(int* A, int N) {
    int i =  blockIdx.x * blockDim.x + threadIdx.x; //berekend de plaats gebruik
        makende van de blocknr*groote block+ thread in block, anders doen de 2
        blokken exact het zelfde
    if (i < N){
        for (int j = 0; j < log2f(N); j++){
            if (threadIdx.x % static_cast<int>(pow(2, j)) == 0) {
            A[i] = max(A[i], A[i+static_cast<int>(pow(2, j))]);
            }
        __syncthreads();
        }
    }
}

int main() {
    int N = 2047;
    int A[N];
    int result[N];

    for (int i = 0; i < N; i++) {
        A[i] = rand() % 100; // Random number between 0 and 99
    }

    for (int i = 0; i < N; i++) {
        printf("%d␣", A[i]);
    }
    printf("\n");

    int *d_A;
    cudaMalloc(&d_A, N * sizeof(int));      //maakt geheugen aan op gpu voor A
    cudaMemcpy(d_A, A, N * sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 1024;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    cudaEvent_t start , stop ;
    cudaEventCreate (& start ) ;
    cudaEventCreate (& stop ) ;
    cudaEventRecord ( start ) ;

    findMax<<<blocksPerGrid, threadsPerBlock>>>(d_A, N);

    cudaDeviceSynchronize();
    cudaEventRecord ( stop ) ;
    cudaEventSynchronize(stop);
    float milliseconds ;
    cudaEventElapsedTime (& milliseconds , start, stop) ;

    cudaMemcpy(result, d_A, N * sizeof(int), cudaMemcpyDeviceToHost);        //
        kopieert array GPU naar B

    printf("Maximum␣value␣is:␣%d\n", result[0]);
    printf("time␣is:␣%f\n", milliseconds);
    cudaFree(d_A);

    return 0;
}
```

Code voor grotere arrays met GPU

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <math_constants.h>

__global__ void findMax(int* A, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        for (int j = 0; j < log2f(N); j++) {
            int offset = static_cast<int>(powf(2, j));
            if ((i % (2 * offset)) == 0 && (i + offset) < N) {
                A[i] = max(A[i], A[i + offset]);
            }
            __syncthreads();
        }
    }
}

int main() {
    const int N = 4096;
    int A[N];
    int result[N];

    for (int i = 0; i < N; i++) {
        A[i] = rand()% 100;
    }

    for (int i = 0; i < N; i++) {
        printf("%d␣", A[i]);
    }
    printf("\n");

    int *d_A;
    cudaMalloc(&d_A, N * sizeof(int));

    cudaMemcpy(d_A, A, N * sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    findMax<<<blocksPerGrid, threadsPerBlock>>>(d_A, N);
    cudaDeviceSynchronize();

    cudaMemcpy(result, d_A, N * sizeof(int), cudaMemcpyDeviceToHost);

    int h_maxVal = INT_MIN;
    for (int i = 0; i < blocksPerGrid; i++) {
        h_maxVal = max(h_maxVal, result[i * threadsPerBlock]);
    }

    printf("Maximum␣value␣is:␣%d\n", h_maxVal);

    cudaFree(d_A);

    return 0;
}
```